

Perfect — here's your **clean Week 1 topic breakdown** with clear **main topics** and **subtopics** for each. This gives you a checklist to learn, practice, and revise before you move to Week 2.

☐ **WEEK 1 – LLM & RAG Foundations**

☐ **1. LLMs (Large Language Models) – Fundamentals**

Subtopics:

- What is an LLM?
 - How LLMs are different from traditional NLP
 - Pretraining vs fine-tuning
 - Role of datasets in LLM training
 - Prompt + context window
 - Tokens and token limits
 - Parameters (size of models)
-

☐ ☐ **2. Transformers Architecture**

Subtopics:

- Why transformers? (Vs RNN/LSTM)
 - Encoder, decoder blocks
 - Self-attention mechanism
 - Q, K, V matrices
 - Multi-head attention
 - Positional encoding
 - Layer normalization, residual connections
 - Feed-forward layers
 - Masking (for decoder)
-

☐ **3. Tokenization in LLMs**

Subtopics:

- What is a token?
- Byte Pair Encoding (BPE)

- WordPiece vs SentencePiece
 - How tokenizers split text
 - Subword tokens and vocabulary
 - Token IDs and special tokens ([CLS], [SEP], etc.)
 - Count tokens using tokenizer
-

□ 4. Text Embeddings

Subtopics:

- What is an embedding?
 - Sentence vs word embeddings
 - Embedding dimension (768, 1024, etc.)
 - Vector representation of meaning
 - How LLMs use embeddings internally
 - Static (Word2Vec, GloVe) vs contextual (BERT, GPT)
-

□ 5. Cosine Similarity in NLP

Subtopics:

- What is cosine similarity?
 - Formula for cosine similarity
 - Why cosine distance matters in vector space
 - Angle between embedding vectors
 - Example: comparing similarity between two sentences
 - Use in semantic search & RAG
-

□ 6. RAG – Retrieval-Augmented Generation (Concept)

Subtopics:

- What is RAG?
 - LLM limitations without memory
 - Role of retrieval in LLMs
 - How RAG solves context limitation
 - RAG flow: query → embed → retrieve → generate
 - Use cases: chat with docs, resume matchers, etc.
-

□ 7. Implementing RAG with LangChain

Subtopics:

- What is LangChain?
- Document loaders (PDF, CSV, websites)
- Text splitting / chunking
- Creating embeddings (OpenAI / SentenceTransformer)
- Storing in vector DB (FAISS / Chroma)
- Retrieval logic
- Using LLM to answer with retrieved context
- Basic chain: Load → Embed → Store → Retrieve → Prompt

□ OPTIONAL (for extra depth)

- Attention visualization tools
- OpenAI Playground: check tokens
- Try embeddings using HuggingFace + SentenceTransformer
- Vector similarity via FAISS

Let me know if you want this in a printable **checklist format**, or I can create a **Notion board** / **Trello layout** for tracking your daily completion.

Absolutely! Here's your **Week 2 topic breakdown** with **main topics** and **subtopics** — focused on **Vector Databases & LangChain**, which are key foundations for building fast and scalable GenAI apps.

□ WEEK 2 – Vector Databases & LangChain

□ 1. Vector Databases – Fundamentals

Subtopics:

- What is a vector database?
 - Why traditional databases (SQL, NoSQL) aren't suited for embeddings
 - Concept of high-dimensional vectors (embeddings)
 - Indexing methods: flat, IVF, HNSW
 - Similarity metrics: cosine similarity, dot product, Euclidean
 - Use cases: semantic search, recommendation, document retrieval
-

□ □ 2. FAISS (Facebook AI Similarity Search)

Subtopics:

- What is FAISS?
 - How FAISS handles similarity search
 - Types of indexes (FlatL2, IVF, HNSW, PQ)
 - How to store and query vectors in FAISS
 - Use FAISS with SentenceTransformer
 - Serialize / save FAISS index
 - Integration with LangChain
-

□ □ 3. ChromaDB

Subtopics:

- What is Chroma?
 - Local lightweight vector DB for rapid prototyping
 - Adding documents and metadata
 - Indexing, searching, updating entries
 - Embedding storage and search
 - Using Chroma in LangChain pipelines
-

□ □ 4. Pinecone

Subtopics:

- What is Pinecone?
- Pinecone architecture and indexing

- Cloud-based scalable vector DB
 - Inserting, querying, updating vectors via API
 - Metadata filtering
 - Comparison with FAISS and Chroma
 - Pinecone + LangChain integration
-

□ 5. LangChain Retrievers

Subtopics:

- What is a retriever in LangChain?
 - Types of retrievers: vector, self-query, multi-vector, BM25
 - Build retriever chain using FAISS / Chroma / Pinecone
 - Customize retrieval: top_k, similarity_score_threshold
 - Using retriever inside a full RAG pipeline
 - Adding metadata filters in retrievers
-

□ 6. Chat with PDF using LangChain

Subtopics:

- Load PDF using PyPDFLoader or UnstructuredLoader
 - Split into chunks using RecursiveCharacterTextSplitter
 - Create embeddings
 - Store in vector DB (FAISS, Chroma, etc.)
 - User asks question → system fetches related chunks
 - Use OpenAI/GPT to answer using the retrieved data
 - How RAG improves over normal chatbot
-

□ 7. Storing & Managing Embeddings

Subtopics:

- What is an embedding store?
- Structure of embedding (vector + metadata)
- Add/update/delete documents
- Vector DB vs embedding in memory
- Persistent storage with FAISS / Chroma
- Saving and loading indexes for future queries
- Embedding search performance tuning

☐ **OPTIONAL ADVANCED (if time permits)**

- Compare vector DBs: FAISS vs Pinecone vs Weaviate vs Chroma
- How indexes affect search performance
- Use LangChain `MultiQueryRetriever` and `SelfQueryRetriever`
- Add filtering by tags (e.g., doc type, date, author)

Would you like:

- A **visual Notion-style tracker** for this week?
- A **practice checklist** for Thursday–Friday?
- A **unique GitHub mini-project idea** to build on Saturday–Sunday for Week 2?

Let me know — I'll prepare it right away!

Perfect — here is your detailed breakdown for **Week 3**, focused on **IBM watsonx.ai + Granite Models**. This week introduces enterprise-grade AI development — key for building large-scale, production-ready LLM apps.

☐ **WEEK 3 – IBM watsonx.ai + Granite Models**

☐ **1. Introduction to IBM watsonx.ai**

Subtopics:

- What is IBM watsonx.ai? (platform overview)
 - Key components: AI Studio, Prompt Lab, Model Inventory
 - Difference between watsonx.ai vs OpenAI vs Hugging Face
 - Benefits: data privacy, governance, enterprise trust
 - Use cases: chatbots, financial summarization, legal docs, etc.
-

□ 2. Granite Models (IBM's Foundation Models)

Subtopics:

- What are Granite models?
 - Granite vs LLaMA, GPT, Claude — main differences
 - Types of Granite models: text, code, chemistry, time-series
 - Why IBM focuses on **transparent, trusted, and safe** models
 - Model sizes (e.g. Granite 13B, 20B)
 - Language capabilities, multilingual support
-

□ □ 3. watsonx Flow: Low-Code AI Pipeline Builder

Subtopics:

- What is IBM watsonx Flow?
 - Drag-and-drop workflow to build AI tasks
 - Build pipelines using pre-built blocks (load data → transform → predict)
 - No-code vs pro-code blending
 - Scheduling & version control of flows
 - Integration with Watson Machine Learning (WML)
-

□ 4. Deploying LLMs on IBM watsonx.ai

Subtopics:

- Setting up your workspace/project
 - Uploading and fine-tuning a model
 - Testing endpoints (chat, completion)
 - Deployment options: API access, watsonx Assistant, public endpoints
 - Managing versions and scaling inference
 - Monitoring usage and model drift
-

□ 5. watsonx.ai APIs (Programmatic Access)

Subtopics:

- How to interact with watsonx via APIs (REST + Python SDK)
 - Authentication & API keys setup
 - Sending prompts and parameters
 - Handling temperature, top_p, max tokens
 - Response parsing and error handling
 - Comparing watsonx API with OpenAI API usage
-

□ 6. IBM AI Studio Overview

Subtopics:

- AI Studio vs Prompt Lab vs Flow
 - Build, evaluate, deploy — all in one
 - Upload datasets and notebooks
 - Evaluate LLMs using different prompts
 - Track experiments
 - Manage fine-tuning jobs
-

□ □ 7. IBM watsonx vs OpenAI vs Hugging Face

Subtopics:

- Pricing and tokens comparison
 - Privacy and on-prem options
 - Model transparency and explainability
 - Enterprise support
 - Best use cases for each platform
 - Which platform is better for what scenario
-

□ OPTIONAL (Bonus Topics if time permits)

- IBM's Guardrails for Responsible AI
- Data Governance inside watsonx
- Connecting watsonx with IBM Cloud Object Storage
- Creating watsonx-powered apps with FastAPI

Let me know if you'd like:

- ☐ Practice tasks for Thursday–Friday
- ☐ A unique mini project using **IBM watsonx.ai** (hosted + GitHub ready)
- ☐ Resume bullet points after Week 3

Next up → Week 4: Hugging Face Ecosystem. Want me to generate that as well?

Here's your detailed breakdown for **Week 4 – Hugging Face Ecosystem**, one of the most powerful platforms for working with open-source LLMs, fine-tuning, deploying, and sharing AI models.

☐ **WEEK 4 – Hugging Face Ecosystem**

☐ **1. Hugging Face Basics**

Subtopics:

- What is Hugging Face?
 - Overview of the Transformers library
 - ☐ Hub: models, datasets, and spaces
 - Installing Hugging Face libraries (`transformers`, `datasets`)
 - Community and open-source ecosystem
 - Role in GenAI (compared to OpenAI, IBM, etc.)
-

☐ **2. Transformers Pipeline API**

Subtopics:

- What is a pipeline in HF?
- Text classification, summarization, translation, Q&A, generation

- Quick use with:
 - `from transformers import pipeline`
 - Plug-and-play model loading with 1 line
 - Switching models and tasks (e.g., GPT-2, BERT, Falcon, Mistral)
-

□ 3. Tokenizers

Subtopics:

- What is a tokenizer in HF?
 - Pretrained tokenizers (`bert-base-uncased`, `gpt2`, etc.)
 - BPE, WordPiece, SentencePiece tokenizers
 - Tokenizing and detokenizing text
 - Customizing and training your own tokenizer
 - Using `AutoTokenizer`
-

□ 4. Hugging Face Datasets Library

Subtopics:

- What is `datasets` library?
 - Loading datasets with:
 - `from datasets import load_dataset`
 - Filtering, preprocessing, and shuffling
 - Common datasets: SQuAD, IMDb, AG News, WikiText
 - Creating your own dataset for fine-tuning
 - Working with large datasets and streaming
-

□ 5. Fine-Tuning Models (BERT / LLaMA / GPT)

Subtopics:

- Difference between pretraining and fine-tuning
 - HF Trainer API vs LoRA/PEFT (Parameter-Efficient Fine-Tuning)
 - Fine-tune BERT for text classification
 - Using `Trainer` or `accelerate` for training
 - Saving and pushing model to Hugging Face Hub
 - Tracking training metrics with Weights & Biases
-

□ 6. Model Hub – Hosting & Sharing Models

Subtopics:

- Upload your model using `transformers-cli` or `huggingface_hub`
 - Model card creation: README, tags, license
 - Versions and checkpoints
 - Making model public vs private
 - How to search/download someone else's model
-

□ 7. Hugging Face Spaces (Deploy AI apps)

Subtopics:

- What is a Space? (Gradio or Streamlit apps hosted by Hugging Face)
 - How to deploy: `app.py` or `app.ipynb`
 - Hosting chatbot, image classifier, or audio app in minutes
 - Free GPU access for demos
 - Link model → space → live AI product
-

□ OPTIONAL (Bonus Topics)

- Hugging Face Inference Endpoints (host custom LLMs as APIs)
 - Using `Diffusers` library for generative image models
 - Integrating Hugging Face with LangChain or FastAPI
 - Using PEFT (LoRA, QLoRA) for cheaper fine-tuning
-

Would you like me to help with:

- □ Practice tasks for Thursday–Friday
- □□ A unique weekend mini-project using HF Spaces + Transformers
- □ GitHub README + resume bullet points

Let me know, and I'll set that up immediately. Also, ready to continue to **Week 5 – GenAI APIs (OpenAI, Gemini, Claude, Mistral)** if you'd like!